Arrays in Functions and Matrix Computation

Matteo Fasiolo

Previously

- An array stores a fixed-length sequence of elements of one type.
- An array of length n has valid indices 0 to n-1.
- A loop can use its counter to process each array element.
- Array elements occupy contiguous memory.
- Access outside the valid range has undefined behaviour.

Today

By the end of the lecture, you should be able to:

1. pass an array and its length to a function;
2. predict whether a function changes the caller's array;
3. use read-only input arrays and output arrays;
4. store matrices as arrays;
5. analyse simple vector and matrix algorithms.

Retrieval check

```
double x[] = {2.0, 4.0, 6.0, 8.0};

for (int i = 0; i < 4; i = i + 1) {
    x[i] = x[i] / 2.0;
}
```

1. Which values does `i` take?
2. What does `x` contain afterwards?
3. What would go wrong if the condition were `i <= 4`?

Recall: a number argument is copied

```
double square(double x) {
    x = x * x;
    return x;
}

int main(void) {
    double value = 2.0;
    double answer = square(value);
    printf("%.1f %.1f\n", value, answer);
    return 0;
}
```

This prints `2.0 4.0`.

Why use a function?

Suppose several parts of a program need a dot product:

$$a \cdot b = \sum_{i=0}^{n-1} a_i b_i.$$

We want one reusable function:

```
double dot(/* two arrays and their length */);
```

We use functions to separate **what** we want to compute from **how** we want to compute it.

But how do we pass arrays to a function?

Consider the problem of printing each values in an array.

Use empty brackets in the parameter declaration:

```
void print_array(double values[], int length) {

    for (int i = 0; i < length; i = i + 1) {
        printf("%.1f ", values[i]);
    }

    printf("\n");

}
```

Call it using the array name and its length:

```
double data[] = {3.1, 2.2, 4.2, 3.2, 0.1};
int length = 5;

print_array(data, length);
```

Note: you need to pass the vector and its length.

Inside this function:

```
void print_array(double values[], int length)
```

`values` does not tell the function how many elements may be accessed.

To call the function correctly:

- `values` provides the elements;
- `length` specifies the valid range;
- the function accesses indices 0 to `length - 1`.

Back to the dot product example:

$$a \cdot b = \sum_{i=0}^{n-1} a_i b_i.$$

This is computed by:

```
double dot(double a[], double b[], int length) {
    double total = 0.0;

    for (int i = 0; i < length; i = i + 1) {
        total = total + a[i] * b[i];
    }

    return total;
}
```

Both arrays must contain at least `length` elements.

NOTE: An array is not copied

Consider the function:

```
void add_one(int values[], int length) {
    for (int i = 0; i < length; i = i + 1) {
        values[i] = values[i] + 1;
    }
}
```

Suppose `main` contains this code:

```
int data[] = {1, 2, 3, 4, 5};
int length = 5;

add_one(data, length);
```

Now `data` contains the value 2, 3, 4, 5, 6.

Calling `add_one(data, length)` changes elements of `data`.

C does not copy all the array elements when making this call.

Instead, the parameter can access the caller's original elements.

The next lecture on pointers will explain why.

How do we prevent this from happening:

```
void print_array(double values[], int length) {

    for (int i = 0; i < length; i = i + 1) {
        printf("%.1f ", values[i]);
    }
    values[0] = 0;  // <- probably an error

    printf("\n");

}
```

?

By using `const` as here:

```
void print_array(const double values[], int length) {

    for (int i = 0; i < length; i = i + 1) {
        printf("%.1f ", values[i]);
    }
    values[0] = 0;  // <- probably an error

    printf("\n");

}
```

`const` states that `print_array` should not change the array elements.

The program will not compile, because `values[0] = 0` is illegal!

Note that we less worried about

```
void print_array(const double values[], int length) {

    length = 3;  // <- probably an error

    for (int i = 0; i < length; i = i + 1) {
        printf("%.1f ", values[i]);
    }

    printf("\n");
}
```

This might cause an error but it will not overwrite the value of `length` outside `print_array`.

Anyway, we can prevent this by:

```
void print_array(const double values[], const int length)
```

Producing an output array

Suppose we want to compute the sum of two vectors $c = a + b$.

C functions cannot return an array type directly.

Instead, the caller can provide an output array:

```
void vector_add(const double a[],
                const double b[],
                double output[],
                int length) {
    for (int i = 0; i < length; i = i + 1) {
        output[i] = a[i] + b[i];
    }
}
```

Here a and b are inputs; output is intentionally modified.

```
double a[3] = {1.0, 2.0, 3.0};
double b[3] = {2.0, 3.0, 4.0};
double c[3];
int length = 3;

vector_add(a, b, c, length);
```

After the call:

```
c = [3.0, 5.0, 7.0]
```

The caller is responsible for providing a sufficiently large output array.

Switch to VS Code

Open `array_functions.c`

The debugger showed that:

- a parameter such as `double x` receives a copy of one value;
- an array parameter allows access to the caller's elements;
- the function needs a separate length;
- `const` states that an input array should not be modified.

We can also identify three different patterns:

Pattern	Example	Array behaviour
Read-only input	`dot(a, b, n)`	elements are read
Modified input	`add_one(a, n)`	elements of `a` change
Output array	`vector_add(a, b, c, n)`	results stored in `c`

You should try to make the intended pattern clear via:

- function names
- use of `const`
- parameter names
- comments

C arrays can represent mathematical vectors.

Many kinds of data are naturally arranged in rows and columns.

E.g., a brain scan is stored as a rectangular grid of values.

How can we work with such data in C?

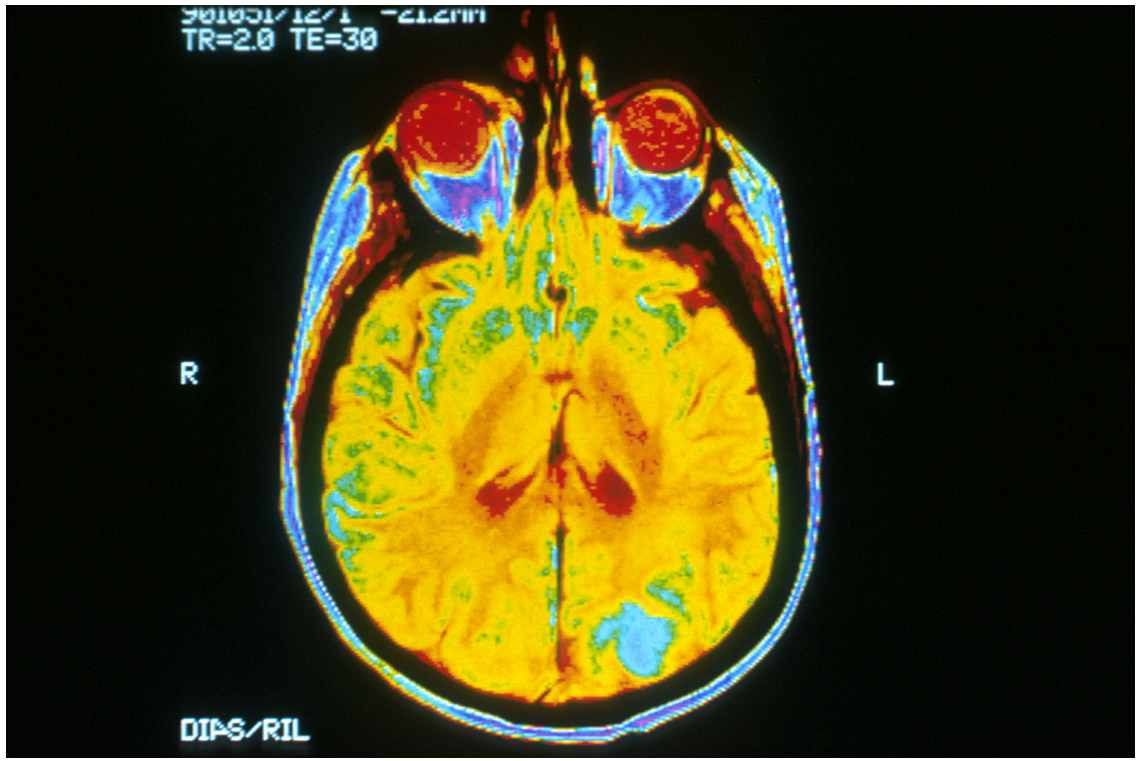


Image source.

Mathematically, gridded data can be represented by matrix.

A matrix is a two-dimensional arrangement of values:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Trick: We can store a matrix in a one-dimensional array.

This is called **flattening** the matrix.

We can do this either in **Row-major** or in **Column-major** order.

Row-major order stores one complete row after another:

```
int values[] = {1, 2, 3, 4, 5, 6};
```

Let's assume A has `nrow` rows and `ncol` columns.

Suppose we use zero-based indexing for matrix elements:

$$A = \begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \end{bmatrix},$$

Element is $A_{i,j}$ with:

- i going from 0 to `nrow-1`
- j going from 0 to `ncol-1`

How to access matrix element $A_{i,j}$ using array indices?

That is $A_{i,j} =$ `values[???]`.

Call $\mathrm{index}_{\mathrm{row}}(i, j)$ the function that gives us the right index.

So that $A_{i,j} = \texttt{values}[\mathrm{index}_{\mathrm{row}}(i, j)]$.

We have

$$\mathrm{index}_{\mathrm{row}}(i, j) = i \times \mathsf{ncol} + j.$$

because

1. increasing `i` skips complete rows;
2. each row contains `ncol` elements;
3. move `j` elements into the selected row.

E.g., for a 2 by 3 matrix

$$A = \begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \end{bmatrix},$$

we find $(A_{1,2})$ using:

$$\text{index}_{\text{row}}(1,2) = 1 \times \text{ncol} + 2 = 1 \times 3 + 2 = 5.$$

Let's check that the mapping

$$\text{index}_{\text{row}}(i, j) = i \times \text{ncol} + j.$$

works.

Consider a matrix with `nrow = 3` and `ncol = 4`:

$$A = \begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \end{bmatrix}.$$

What is the flat row-major index of:

1. $(A_{0,0})$?
2. $(A_{0,3})$?
3. $(A_{1,0})$?
4. $(A_{2,3})$?

Column-major order

Column-major order stores one complete column after another.

For example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

is stored into

```
int values[] = {1, 4, 2, 5, 3, 6};
```

The flat index is given by

$$\text{index}_{\text{column}}(i, j) = j \times \text{nrow} + i.$$

NOTE: C conventionally uses row-major order; R stores matrices in column-major order.

Printing a row-major matrix

```
void print_matrix(const int values[], int nrow, int ncol)
{
    for (int i = 0; i < nrow; i = i + 1) {
        for (int j = 0; j < ncol; j = j + 1) {
            printf("%d ", values[i * ncol + j]);
        }
        printf("\n");
    }
}
```

Outer loop moves across rows; inner loop across its columns.

Exercise: how would you print a column-major matrix?

Live demonstration 2

Switch to VS Code

Open `matrix_layout.c`

Both arrays represented the same matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

via

```
row-major storage:      1 2 3 4 5 6
column-major storage:   1 4 2 5 3 6
```

Notes on time complexity

For an array of length `n`:

- `sum`, `dot`, `add_one` and `vector_add` are all $O(n)$.

For a matrix with `m` rows and `n` columns:

- printing visits every one of the mn entries;
- the time complexity is $O(mn)$.

The nested loops do not automatically imply $O(n^2)$.

Here the `for()` loops are over n and m.

Check your understanding

1. Why does a function with an array input normally need also a length parameter?
2. What does `const` say about an array parameter?
3. Why can `add_one(a, n)` change the elements of `a`?
4. What is the row-major index of $A_{i,j}$?
5. How many iterations are needed to print an $(m \times n)$ matrix?

Conclusion

1. Pass an array together with the length to functions.
2. C functions do not copy array elements for an array parameter.
3. Use `const` for read-only arrays and an explicit output array for multiple results.
4. C functions cannot return arrays. If you want to store array results you need to have an array parameter where to store results (see `vector_add` in `array_functions.c`).
5. Row-major index: `i * ncol + j`.
6. Column-major index: `j * nrow + i`.